

ai-deck-compiler

AI가 기획한 의미 구조를 editable PowerPoint로 컴파일하는 presentation engine

Semantic Blueprint Planning · Deterministic Rendering · Preset Showcase

ai-deck-compiler

오늘 볼 것

01

AI가 만든 PPT가 왜 다시 손봐야 하는 산출물이 되는가

02

blueprint-first workflow가 내용을 구조화하는 방식

03

짧은 기간에 쌓은 compiler, skill, preset, example evidence

04

같은 deck을 teal, vivid, modern preset으로 재생성하는 결과

05

이 showcase 자체가 public proof artifact가 되는 이유

01

Why It Exists

01

AI가 만든 deck도 구조가 없으면 다시 수작업이 된다

- 일반적인 AI slide 생성은 빠르지만, 내용의 의미 구조와 편집 가능한 객체 경계가 쉽게 사라진다.
- 차트, 표, 아키텍처, 의사결정, 코드 예시는 모두 다른 표현 방식이 필요한데 bullet slide 하나로 놀리기 쉽다.
- `ai-deck-compiler`는 AI가 고른 표현 결정을 `blueprint.yaml`에 남기고, renderer가 PowerPoint 객체로 재현한다.
- 검증 명령, preview 이미지, tracked result files가 산출물을 반복 가능한 작업 단위로 만든다.

Prompt output이 아니라 구조화된 artifact를 남긴다

AI-ONLY SLIDE DRAFT

- 결과물이 이미지나 느슨한 bullet 묶음으로 남기 쉽다.
- 같은 요청을 다시 실행하면 layout과 강조 수준이 흔들릴 수 있다.
- 리뷰는 “보기 괜찮은가” 중심이라 수정 근거가 약하다.
- 코드·명령어·재생성 절차가 일반 문장처럼 섞인다.

BLUEPRINT-FIRST DECK

- slide type, chart data, diagram node, notes가 YAML에 보존된다.
- 같은 blueprint와 preset이면 같은 rendering 규칙으로 재생성된다.
- validate, deck, preview, review loop가 품질 게이트가 된다.
- code block은 boxed monospace component로 분리해 보여준다.

내용을 그대로 옮기지 않고 표현 방식을 선택한다

BULLET TRANSLATION

- × 수치, 결정, 구조, 절차가 모두 비슷한 bullet로 평평해진다.
- × 슬라이드 제목은 주제 라벨이 되고 결론은 본문에 묻힌다.
- × 차트나 diagram은 나중에 사람이 다시 설계해야 한다.

SEMANTIC BLUEPRINT

- ✓ 숫자는 KPI/chart, 관계는 architecture, 순서는 flow로 승격한다.
- ✓ action title, callout, recommendation, takeaways가 강조 계층을 만든다.
- ✓ renderer는 의미 필드를 읽어 editable PPTX 객체로 출력한다.

작은 repo지만 deck 생성에 필요한 핵심 블록을 갖췄다

16

▲ core, data, diagram

Slide Types

59

▲ parser/render/snapshot

Tests

3

▲ teal/vivid/modern

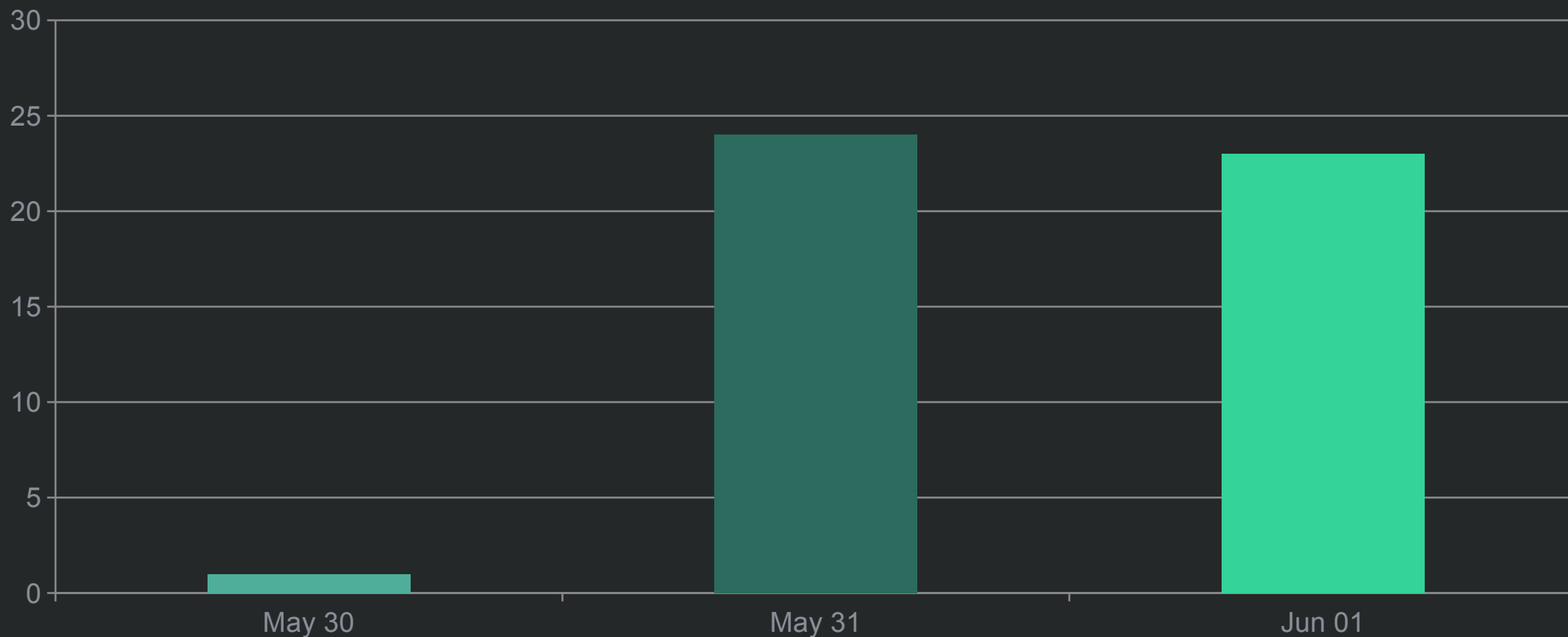
Presets

10

blueprint/PPTX/PDF/gallery

Result Files

집중적인 구현이 compiler와 workflow를 동시에 밀어 올렸다



제품은 코드, skill, 예제, 문서를 함께 갖춘다

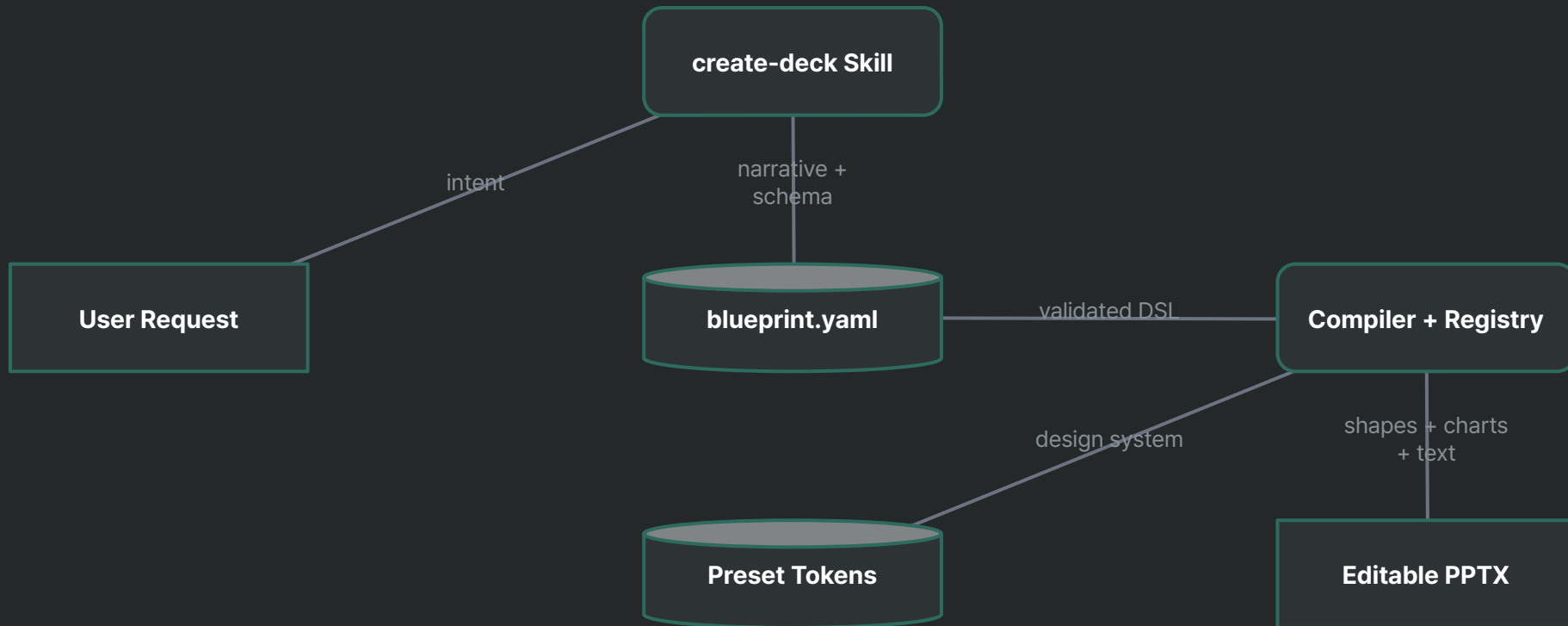


02

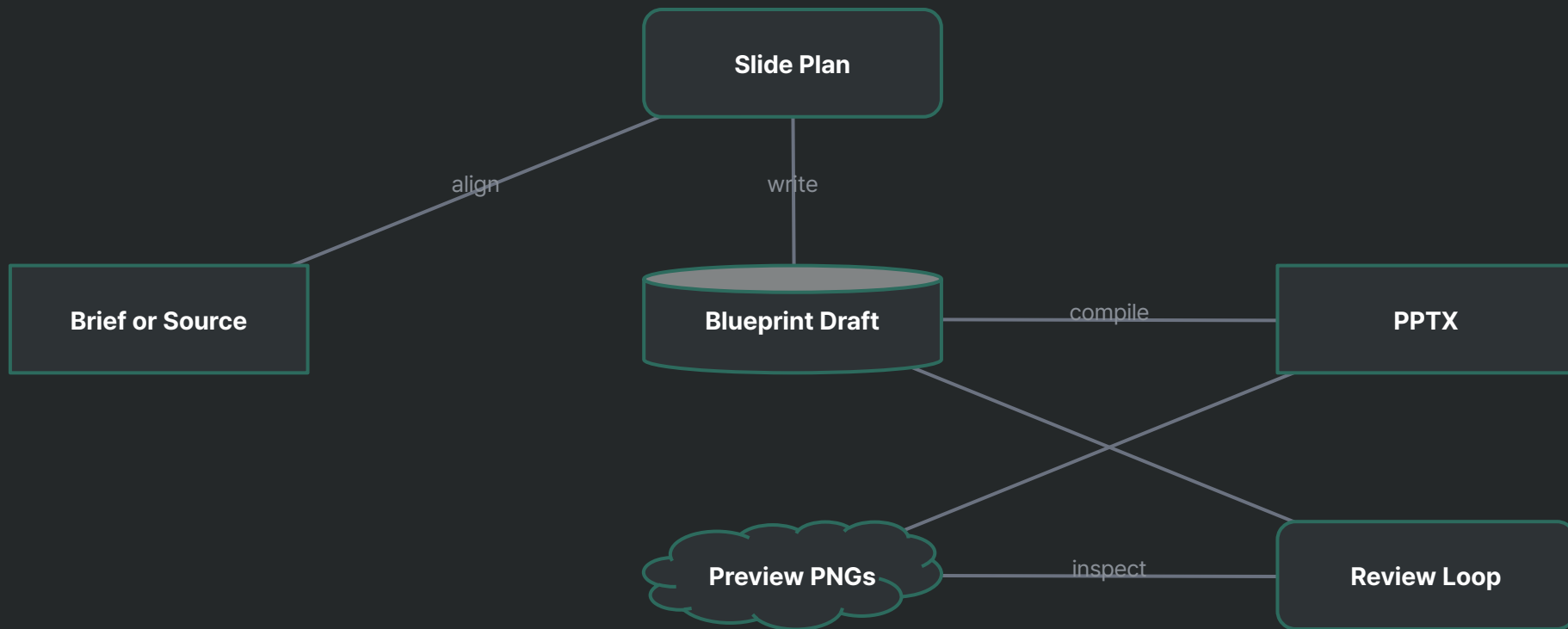
How It Works

02

AI planning과 deterministic rendering을 분리한다



작성과 검토는 preview gate를 통과하며 반복된다

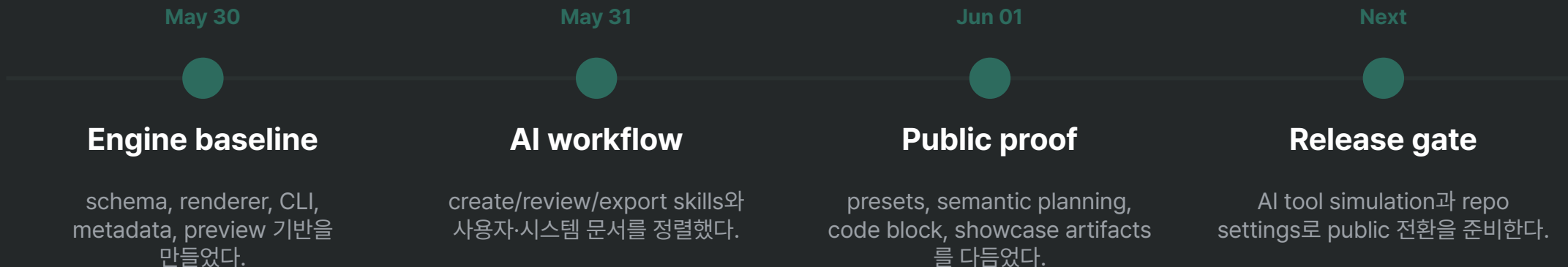


Preview는 자동 종료점이 아니라, 사람이 결과를 보고 조정하는 품질 게이트다.

같은 content, 세 가지 presentation tone

Preset	Theme	Best fit	What this showcase proves
teal	dark	AI-native technical briefing	차분한 executive tone과 callout bar
vivid	dark	Product demo / feature showcase	강한 contrast와 vivid accent
modern	light	Business report / sharing doc	office-friendly light document tone

3일의 작업은 기능보다 workflow 완성에 집중됐다



대표 예제는 제품 자체를 증명해야 한다

Generic business sample

- ✓ 누구나 익숙한 주제로 읽기 쉽다
- ✓ 도메인 설명 부담이 낮다
- × 이 repo가 왜 필요한지 첫 화면에서 충분히 말하지 못한다
- × preset, slide type, code block, workflow 증거가 분산된다

Product proof showcase

- ✓ 제품 메시지와 산출물 품질을 같은 deck에서 보여준다
- ✓ chart, architecture, code block, preset 비교가 맥락 안에 놓인다
- × content 품질 기준이 일반 sample보다 높아진다

▶ examples/results는 generic sample이 아니라 ai-deck-compiler 자체를 설명하는 product proof artifact로 둔다.

이 deck은 제품 설명이자 출력 품질 검증이다

- AI는 발표 목적, 청중, source를 읽고 slide type과 강조 계층을 결정한다.
- Compiler는 blueprint와 preset token을 읽어 chart, table, diagram, code block을 PowerPoint 객체로 만든다.
- Preview와 review loop는 "생성됨"에서 멈추지 않고 사람이 품질을 확인하는 단계로 만든다.

Key Takeaways

- ✓ 자연어 요청은 검증 가능한 `blueprint.yaml`로 남는다
- ✓ 같은 구조를 teal, vivid, modern preset으로 재생성할 수 있다
- ✓ showcase deck 자체가 public release 전 proof artifact가 된다

Regeneration commands

BASH

```
BP=examples/results/showcase-teal-dark.blueprint.yaml
OUT=examples/results/showcase-teal-dark.pptx
PDF=examples/results/showcase-teal-dark.pdf
npm run validate -- --blueprint "$BP"
npm run deck -- --blueprint "$BP" --output "$OUT"
npm run export-pdf -- "$OUT" --out "$PDF"
npm run preview -- "$OUT" --out temp/showcase-teal-preview --dpi 120
```

- vivid/modern 결과물은 같은 명령에서 파일명과 `--out` 경로만 바꿔 재생성한다.
- 이 slide는 최근 추가한 fenced code box와 syntax highlighting을 실제 showcase 안에서 보여준다.

ai-deck-compiler showcase

From prompt to proof artifact

AI가 만든 초안을 넘어, 재생성 가능하고 편집 가능한 presentation system으로 남긴다